Equations & User Flow

Hurdle Model Equations

Stage 1 — Logit `log(P/(1-P)) = f(X)`

Gradient-boosted classifier fits a log-loss objective over all windows. Output is a raw score converted to `P(crash_occurs)`.

- `P` = crash probability
- `f` = learned tree ensemble
- `X` = feature matrix

Isotonic Calibration `P_cal = isotonic(P_raw)`

Corrects the classifier's output so it reads as a true probability. Fitted on the held-out validation set to fix tree-classifier overconfidence.

- `P_cal` = calibrated probability
- `P_raw` = raw classifier score

Stage 2 — Poisson Count `log(E[y|crash]) = f(X)`

Poisson-loss regressor trained **only on crash windows**. Output is expected crash count given a crash occurred.

- `y` = crash count
- `f` = learned tree ensemble
- `X` = feature matrix

Hurdle Combination `λ = P(crash) × E[count|crash]`

Multiplies both stages into one expected crash rate per window per segment. Core prediction of the model.

- λ = expected crashes/window
- $P(\text{crash})$ = Stage 1 output
- $E[\text{count}|\text{crash}]$ = Stage 2 output

Tail Weight $w_{tail} = 1 + \alpha \times \log(1 + y)$

Upweights rare high-count windows so the optimizer doesn't ignore 5-crash events in a sea of 1-crash windows. Applied when $y \geq 2$.

- w_{tail} = tail emphasis weight
- α = emphasis strength (2.0)
- y = crash count

Final Sample Weight $w_{final} = w_{samp} \times w_{tail}$

Merges the sampling correction weight with tail emphasis. Normalized to preserve gradient scale, then capped at 50.

- w_{final} = combined training weight
- w_{samp} = sampling correction weight
- w_{tail} = tail weight

Lambda Cap $\lambda = \text{clip}(\lambda, 0, 50)$

Stability cap — prevents extreme predictions on rare segments from dominating routing math.

- λ = predicted crash rate

Poisson Window Probability $P_{raw} = 1 - e^{-\lambda}$

Converts raw λ into a window-level crash probability (Poisson CDF). Used as the calibration target.

- P_{raw} = uncalibrated crash probability
- λ = expected crashes/window

Poisson Deviance $D = 2 \times \text{mean}(\hat{y} - y + y \times \log(y/\hat{y}))$

Evaluation metric. Measures how well the predicted count distribution matches actual counts.

- D = deviance score
- $\hat{y}$ = predicted count
- y = actual count

Routing Engine Equations

Travel Time $t = len\_m / (speed\_kmh / 3.6) / 3600$

Converts a segment's physical length and assumed road-class speed into hours. Base edge weight for the fastest route.

- t = travel time (hours)
- len_m = segment length (meters)
- $speed\_kmh$ = road-class speed

GPS Snap Distance $dist \approx \Delta deg \times 111{,}000$

Flat-earth approximation to snap a GPS coordinate to the nearest graph node. Rejects snaps beyond 300 m.

- $dist$ = approx distance (meters)
- Δdeg = degree difference in lat or lon

Combined Multiplier $m = m\_weather \times m\_time$

Multiplies weather and time-of-day factors into one scalar applied to all λ values before routing.

- m = combined multiplier
- $m\_weather$ = weather factor
- m_time = time-of-day factor

Adjusted Lambda $\lambda\_adj = \lambda \times m$

Scales each segment's predicted crash rate up for hazardous conditions (e.g., snow + rush hour = 1.875×).

- λ_{adj} = adjusted crash rate
- λ = model-predicted crash rate
- m = combined multiplier

Expected Crashes $E = \lambda_{adj} \times t$

Rate × time gives a dimensionless expected crash count for one traversal of the segment.

- E = expected crashes
- λ_{adj} = adjusted crash rate
- t = travel time (hours)

Risk Weight $w_{risk} = t + \beta \times E$

Adds a crash penalty to travel time, making riskier segments appear "slower" to Dijkstra.

- w_{risk} = risk-penalized edge weight (hours)
- t = travel time
- β = penalty scalar (0.1 hrs/crash)
- E = expected crashes

Route Total Expected Crashes $\Lambda = \Sigma\ (\lambda_i \times t_i)$

Sums expected crashes across every edge in a route to get total trip-level crash exposure.

- Λ = total expected crashes
- λ_i = adjusted crash rate for edge i
- t_i = travel time for edge i

Route Crash Probability $P = 1 - e^{-\Lambda}$

Poisson CDF: probability of at least one crash on the full route. Shown as `routeProbability` in the API response.

- P = crash probability
- Λ = total expected crashes

Risk Label Thresholds $\lambda \leq p70 \rightarrow low$ / $p70 < \lambda \leq p90 \rightarrow medium$ / $\lambda > p90 \rightarrow high$

Assigns color labels relative to the city-wide λ distribution, not arbitrary fixed values.

- `λ` = segment crash rate
- `p70` = 70th percentile of all segments
- `p90` = 90th percentile of all segments

User Flow in Plain English

Before you open the app, the server pre-loads the Toronto road network into a graph, assigns travel times to every edge based on road class, and runs the hurdle model to get a λ (crash rate) for every segment — all cached in memory.

When you search for a route, your origin and destination GPS points are snapped to the nearest graph nodes. The server then adjusts every segment's λ upward based on current weather and time of day.

Two routes are computed with Dijkstra simultaneously — one minimizing raw travel time, one minimizing a risk-penalized travel time where dangerous segments look artificially slower.

Each route is scored by summing λ × time across all its edges to get total expected crashes (Λ), then converting that to a crash probability with 1 − e^{−Λ}.

The map renders both routes with per-segment color coding (green/yellow/red) based on how each segment's λ compares to all of Toronto — so a "high risk" label means that segment is in the worst 10% city-wide, not just high in absolute terms.